Can Agents Learn When to Verify or Repair Memory Under Budget Constraints and Access Uncertainty?

Arik Gershman
Department of Computer Science
University of Maryland, College Park
College Park, MD, USA
arikger@umd.edu

Shohini Rhea Sarkar
Department of Computer Science
University of Maryland, College Park
College Park, MD, USA
ssarkar6@terpmail.umd.edu

Amelia Harn
Department of Computer Science
University of Maryland, College Park
College Park, MD, USA
aharn1@umd.edu

***Abstract*—Long-running LLM agents store derived memories, such as summaries built from many raw facts, so they do not have to reread their full history on every query. When a fact is corrected, every summary built on it becomes stale. Existing memory systems focus on detecting or invalidating outdated facts, but leave open what to do with the stale summaries afterward. We study this as an online decision made at query time: when a query reaches a stale summary, the agent can either verify it against current source facts for that query alone, at a small cost paid every time, or repair it once at a larger cost that benefits all later queries. Because the number of future queries is unknown, this is a rent-or-buy problem. We build a memory system with three fixed pieces: an append-only fact ledger that never deletes information, summaries that record which facts they were built from, and retrieval that blends keyword and semantic search. We then compare maintenance policies (simple threshold rules with worst-case guarantees), and a lightweight learned policy that predicts how often a stale summary will be queried again, against three baselines: always repair eagerly, never repair (verify forever), and a hindsight-optimal oracle that knows the future. We measure total maintenance cost, how often answers come from stale facts, p95 query latency, and how far each policy falls short of the oracle, testing on controlled synthetic workloads at 10K and 100K memories under limited repair budgets and shifting demand, and on real-history question-answering benchmarks.**

We build a fixed memory substrate with an append-only fact ledger, dependency-tracked summaries, and hybrid retrieval, and compare threshold policies with worst-case guarantees and a lightweight learned policy that predicts reuse against eager, lazy, and hindsight-optimal baselines. We evaluate total cost, stale-answer rate, p95 latency, and regret on controlled synthetic workloads at 10K and 100K memories, under repair budgets and shifting demand, and on real-history QA benchmarks.

***Index Terms*—agent memory, memory maintenance, staleness, online algorithms, ski rental, learning-augmented algorithms, retrieval-augmented generation**

I. Problem

LLM agents that run for weeks or months keep long-term memory in two layers: raw records of what they have been told, and derived memories, such as summaries, user profiles, and reflections, that are generated from those records and retrieved at query time. Derived memories make retrieval cheaper and answers more coherent, but they go out of date. When a user corrects a fact ("I moved to Boston," "the deadline is now Friday"), every summary built on the old value is wrong, and the agent may keep answering from it.

Recent systems address the detection side of this problem. Graphiti, the engine behind Zep, stores facts in a temporal knowledge graph and invalidates edges when newer information contradicts them [4], and All-Mem [2] and freshness-aware retrieval methods [3] likewise track or down-weight outdated content. What these systems leave open is what to do once staleness is known. One option is to rebuild every affected derived memory as soon as a correction arrives. This wastes work on memories that are never queried again, and the waste grows with memory size and with the number of summaries each fact feeds. The other option is to check the source records every time a stale memory is used. Each check is cheap, but the memory is never fixed, so a popular summary pays for the same check again and again.

We treat this as an online choice made at query time. When a query touches a stale summary, the agent can VERIFY, consulting the current source facts to answer this one query at cost c_v, or REPAIR, regenerating the summary at a larger one-time cost c_r that benefits every later query. If the summary will be queried k more times before its sources change again, repair is worth it exactly when $k \cdot c_v > c_r$, but k is unknown when the decision is made. This is the rent-or-buy structure of the ski-rental problem [5], which gives us simple policies with worst-case guarantees and a clear place where learned predictions of reuse can help [7].

Our research question is: *when a correction leaves part of a large memory system potentially stale, can an agent decide at query time whether to temporarily verify it against fresh source evidence or permanently repair it, and can it make this decision under a limited maintenance budget without knowing how frequently that memory will be queried again?* Our verify-or-repair hypothesis is that a policy which bases this decision on expected reuse lowers total maintenance cost compared with always repairing or always verifying, without raising the stale-answer rate, and stays close to a hindsight-optimal policy when the repair budget is limited.

II. Related Works

This work is at the intersection of two areas: online decision making under uncertainty and memory maintenance for agents with long histories. The field of online decision making provides our theoretical framework, while research on memory maintenance informs our problem setting and baselines.

A. Online Rent-or-Buy Decisions

The verify-versus-repair choice we study has strong parallels to the ski rental problem: when should an agent keep renting skis (multiple smaller costs) versus purchasing a pair (one larger one-time cost)? Karlin et al. [5], [6] introduced this framework and analyzed deterministic and randomized threshold policies with competitive guarantees. This resembles our setting, where we must choose between verifying and repairing without concrete knowledge of future queries.

Purohit et al. [7] later used machine-learned predictions to improve these bounds. They estimate future rental days with an ML predictor and introduce a parameterized policy that adjusts the buying threshold based on that prediction. The parameter controls the tradeoff between trusting the prediction and falling back on the worst-case threshold. We adopt a similar framing: our learned policy predicts future query counts for stale memory components and uses a comparable consistency-robustness structure to decide whether to verify or repair.

Li et al. [8] extend ski rental by allowing multiple items to be rented separately, bought individually, or purchased as a discounted bundle. Their combinatorial formulation may inform how we repair parts of memory—whether to repair related components together or separately.

B. Knowledge Editing in Language Models

A parallel line of research addresses how models can correct outdated knowledge stored in their weights. Meng et al. [9], [10] show that factual associations in GPT models are localized in specific feed-forward layers and apply targeted rank-one updates to edit them. However, this approach corrects the targeted fact but not the downstream conclusions that depend on it. EchoEdit [11] further explores this by editing semantically related nodes to control ripple effects.

C. Memory Systems for Long-Running Agents

Recent agent memory work studies how to store and retrieve information over long horizons. All-Mem [2] treats memory as a dynamic graph and uses an LLM offline to propose split, merge, or update operations. LongMemEval-V2 [13] benchmarks how well agents recall facts about an environment from past trajectories.

D. The Gap to Address

Existing work leaves a gap: after a stale memory is detected, how should we decide whether to repair all associated memories eagerly, reorganize on a fixed schedule, or simply verify indefinitely? None of the existing work asks whether a particular stale component is worth fixing now given uncertainty about how often it will be queried again. Framing this as a rent-or-buy problem gives us a principled way to approach it with support from prior literature. A more thorough literature review can be found in the appendix.

III. Scope and Setup

A. Research Scope

Our goal is to study the maintenance decision on its own. How much verification (c_v) and repair (c_r) cost, and how fast queries run, all depend on four things: how memory is laid out, how staleness is spread to affected summaries, how summaries are retrieved, and which model does the verifying and repairing. We keep all four simple and identical across experiments, so that any difference we measure comes from the policy alone. Throughout this proposal, a *memory component* is one derived summary; the raw facts are stored separately in the ledger.

The scope of the research is defined by the following fixed and variable system components:

***Fixed* Memory Design:**

- *Source Fact Ledger*: A table of facts that only ever grows, modeled after MemStrata [1]. Facts are never deleted or edited in place. When a correction arrives, we add a new record and mark the old one as no longer valid from that point on, so the full history is kept.
- *Derived Summary Store*: Holds the natural-language summaries generated from ledger facts. Each summary keeps a list, `source_facts`, of exactly which fact versions it was built from, plus a few counters the policy uses. A lookup table (an inverted index) maps each fact to every summary built from it. When a fact is corrected, the system uses this table to mark every summary that depends on it as stale (`is_stale = true`). Repair rewrites a summary using the current versions of its facts and updates `source_facts` to match; the ledger itself is never changed.

***Variable* Policy:**

Whenever a query retrieves a summary marked stale, a policy chooses one of two actions:

- VERIFY: Look up the current versions of the summary's facts and use them to answer this one query correctly, without changing the stored summary (cost c_v).
- REPAIR: Rewrite the summary from the current facts, so it is no longer stale for any future query (cost c_r, larger than c_v).

We will implement several policies, ranging from simple threshold rules with guaranteed worst-case cost to small learned models that predict how often a summary will be used again, and compare them against standard baselines (Section IV), using the evaluation described in Section V.

B. Dataset

Standard QA benchmarks do not test the decision we study: facts in them are not corrected over time, and we cannot

control how often a stale summary is asked about afterward. We need workloads where corrections and queries are mixed together over time, and we use two sources.

Controlled Synthetic Workloads (Primary): We generate 10,000 memory components as the base scale, then scale to 100,000 (and, if time permits, 1,000,000) for stress testing. A stream of corrections makes summaries stale. We control how often each stale summary is asked about again by placing it in one of three groups, based on how many queries it receives before it goes stale again (a "stale episode," Section IV-A): cold (0–2 future queries), warm (3–10), and hot (11–50). Because we choose the number of future queries k for every summary, we know the best action in advance and can measure exactly how far each policy is from it.

Real-History Benchmarks (Secondary): After testing policies on synthetic data, we check whether they still work on more realistic patterns of corrections and questions, using the parts of LongMemEval [12] that test reasoning about time and updated knowledge, and RealTime QA [14]. These let us confirm that policies which save maintenance cost do not hurt answer accuracy.

C. Models and Compute

Verification and repair run on local open-weight models of about 7–8 billion parameters, Llama 3.1 8B Instruct [15] or Qwen 2.5 7B Instruct [16]. Running them locally avoids the network delay of hosted APIs, which would distort our latency measurements. Retrieval uses BGE-M3 [17], a locally hosted embedding model that produces, in a single pass, both a meaning-based representation (for semantic search) and word-based weights (for keyword matching). The maintenance policy is a gradient-boosted decision tree model (XGBoost [18] or LightGBM [19]), which works well on table-style data and is far cheaper to run than an LLM. The synthetic data is written by a hosted LLM such as GPT-4o-mini. This happens once, ahead of time, so network delay does not matter, and a hosted model can produce the large amount of varied, good-quality text we need at 100K–1M scale faster and more cheaply than our local 8B models.

Early policy design, small-model testing, and training of the learned policy at the 10K scale will run on Google Colab. For 100K components and beyond, we will move to a PC with a GeForce RTX 5070 that our group has access to. This GPU has 12 GB of memory, which is not enough to hold an 8B model at full (16-bit) precision, so we will use a compressed 4-bit version of the model produced with AWQ [20], served through vLLM [21]. We will use the same compressed model on Colab so that costs measured on the two machines are comparable. If needed, we will request access to UMD's Zaratan HPC cluster.

IV. Methodology

A. Problem Formulation

We organize decisions around *stale episodes*. A stale episode for a summary s is the stretch of time from when a correction marks it stale until the next correction marks it stale again. We use this unit because a repair only pays off until the summary's facts change again; after that, it is stale once more and the decision starts over. Let k_s be the number of queries that reach s during one episode. On each of these queries, the policy chooses between VERIFY and REPAIR. Once s is repaired, the remaining queries in the episode use the fresh summary at no extra cost. Summaries that are not marked stale need no decision and are used directly.

We score a policy π by adding up everything it spends, plus a penalty for each wrong answer:

$$C(\pi) = c_v \cdot N_v(\pi) + c_r \cdot N_r(\pi) + \lambda \cdot N_{\text{stale}}(\pi), \qquad (1)$$

where N_v and N_r are the number of verifications and repairs, N_{stale} is the number of answers that used outdated facts, and λ sets how costly each such answer is. Our own policies always verify or repair a stale summary before using it, so they give stale answers only when the model itself makes a mistake. The penalty matters mostly for baselines such as Periodic Refresh, which knowingly use stale summaries between refreshes.

Looking back after the fact, the best choice for an episode is simple. If the summary is queried often enough that verifying every time would cost more than one repair ($k_s \cdot c_v > c_r$), the best choice is to repair on the first query. Otherwise, it is best to just verify every time.

We do not assume fixed values for c_v and c_r. Instead, we measure them on our own hardware (GPU time and number of generated tokens per operation) and report results in terms of the ratio $\rho = c_r/c_v$, which is simply how many verifications one repair is worth.

B. Memory Substrate

This subsection gives the implementation details of the fixed design in Section III.

Source Fact Ledger. Each record stores an ID (`fact_id`), what the fact is about (`entity`, `attribute`, `value`), when it is valid (`valid_from`, `valid_to`), a `version` number, and which older record it replaces (`supersedes`). The ledger handles four kinds of change: corrections (a wrong value is fixed), retractions (a fact is withdrawn with no replacement), updates over time (a value that was true really does change, such as a new address), and temporary exceptions (a new value that holds only for a limited time, after which the old value applies again). Each one is stored by adding a new record and marking when the old record stopped being valid, so nothing is ever overwritten.

Derived Summary Store. Besides its text and `source_facts` list, each summary stores its embeddings (the numeric representations used for search), its stale flag, the time it became stale, and a few running counts for the policy: how many queries have reached it in the current episode, how much has already been spent verifying it, and how long it has been since its last repair.

Staleness Propagation. When a correction arrives, the system looks up which summaries use the changed fact and marks each one stale. This takes time proportional to the number of affected summaries, and nothing is rewritten yet. If some summaries are built from other summaries, the stale marks are

Fig. 1. Steps taken for each query.

Require: query q, policy π, repair budget B
$S \leftarrow$ HYBRIDRETRIEVE(q) {find relevant summaries}
for all $s \in S$ **do**
  if s.is_stale **then**
    $a \leftarrow \pi$.DECIDE(s, q, B) {verify or repair?}
    if $a =$ REPAIR **and** $B \geq c_r$ **then**
      REPAIR(s); $B \leftarrow B - c_r$ {pay from budget}
    else
      VERIFY(s, q) {this query only}
    end if
    update episode counters of s
  end if
end for
return ANSWER(q, S)

passed down the chain, so a summary of a stale summary is also marked stale. This rule is deliberately cautious: it marks a summary whenever any of its facts changes, even if the change would not alter what the summary says.

Hybrid Retrieval. Each query is turned into an embedding with BGE-M3, and we run two searches: a meaning-based search using FAISS [22], and a keyword search. We combine the two ranked result lists with reciprocal rank fusion [23], a simple method that favors summaries ranked highly by either search, and pass the top n summaries to the policy.

C. Maintenance Operations

VERIFY looks up the current version of every fact in the summary's source_facts, finds the ones that have changed since the summary was written, and gives the LLM both the stale summary and the changed facts so it can answer the query correctly. The stored summary is not changed and stays marked as stale.

REPAIR rewrites the summary from the current facts, recomputes its embeddings, updates both search indexes, records the new fact versions in source_facts, and clears the stale flag. The query is then answered from the repaired summary.

Fig. 1 shows the steps every query follows. The policies differ only in how they make the VERIFY-or-REPAIR choice (the DECIDE step).

D. Maintenance Policies

We implement three policies, from simplest to most involved.

P1: Accumulated threshold. Keep verifying until the total spent on verifications roughly equals the cost of one repair, then repair. Concretely, let $m = \lceil c_r/c_v \rceil$ be the number of verifications that cost as much as one repair. The policy verifies on the first $m-1$ queries of an episode and repairs on the m-th. This is the classic ski-rental strategy ("rent until you have paid the purchase price, then buy"). It needs no prediction, and on every episode it is guaranteed to cost at most about twice as much as the best possible choice made in hindsight [5]. We also test a randomized version, which picks the repair point at random from a specially chosen distribution; on average, it costs at most about 1.58 times the best choice [6].

P2: Predicted threshold. Estimate $\hat{k}_s$, the number of queries the summary will still receive in this episode, by multiplying how often it has been queried recently (a moving average that weights recent queries more) by how long we expect it to be until its facts are corrected again. Repair right away if this estimate says verifying would cost more than repairing ($\hat{k}_s \cdot c_v > c_r$), and verify otherwise. This tests the simple rule from Section IV-A when the number of future queries has to be guessed.

P3: Learned demand-aware policy. A small machine-learning model (gradient-boosted decision trees) predicts $\hat{k}_s$ using information available at the moment of the decision. Its inputs fall into five groups. The first is how the summary has been used: queries in this and earlier episodes, time since the last query, and the average gap between queries. The second is the correction that made it stale: what kind of change it was, how many of its facts changed, and how long ago. The third is what the summary depends on: how many facts it uses and how often those facts have been corrected before, since a summary built on frequently changing facts is likely to go stale again soon after a repair. The fourth is how expensive it is to repair, which depends mostly on the summary's length. The fifth is how likely the stale text is to give a wrong answer, measured by how different the old and new fact values are in meaning. The model is trained on separate example traces where the true k_s is known.

A policy that relies only on its prediction can do badly when usage patterns change, so we also test a hedged version following Purohit et al. [7]. When the model predicts many future queries ($\hat{k}_s \geq m$), it repairs early, at query $\lceil \gamma m \rceil$; when it predicts few, it waits longer, until query $\lceil m/\gamma \rceil$. The setting γ, between 0 and 1, controls how much we trust the model: smaller values follow the prediction more closely, and $\gamma = 1$ ignores it and behaves like P1. If the predictions are accurate, the cost stays within a factor of $1+\gamma$ of the best choice; even if the predictions are completely wrong, it stays within a factor of $1 + 1/\gamma$.

Repair budget. Repairs are paid from a budget B that resets at the start of each time window. When the budget runs out, the system verifies instead, so answers stay correct, and adds the skipped repair to a waiting list sorted by how much each repair is expected to save ($\hat{k}_s c_v - c_r$, the predicted verification cost avoided minus the repair cost). When budget becomes available again, the repairs expected to save the most are done first.

E. Baselines and Oracle

We compare against five simple baselines. *Eager Refresh* repairs every affected summary as soon as a correction arrives. *Periodic Refresh* repairs all stale summaries every T steps and uses stale summaries as-is in between. *Repair-on-First-Access* repairs a stale summary the first time it is queried. *Verify-Only* never repairs. *Oldest-First* spends the repair budget in

the background on the summaries that have been stale the longest.

Because our system marks a summary stale whenever any of its facts changes, some of those marks are false alarms. To count them, we use a *semantic dirty-bit* check at the toy and 10K scales: we rewrite each marked summary, ask both versions the same set of test questions, and count it as truly stale only if the answers differ. This check is too expensive to use as a real policy, but it shows how much work each policy spends on summaries that did not actually need fixing.

The *hindsight oracle* sees the entire future stream of queries and corrections and makes the best choice for every episode. It cannot exist in practice, but it gives us a best-case reference. With a limited repair budget, the oracle must also choose which episodes are most worth repairing in each budget window. This is the classic knapsack problem (fitting the most value into a fixed capacity), which we solve exactly at 10K and approximately at larger scales, by repairing first the episodes with the most savings per unit of cost. We define a policy's *regret* as how much more it spends than the oracle: $C(\pi) - C(\text{oracle})$.

F. Synthetic Workload Generator

The generator creates entities (such as people or projects) with typed attributes, and builds each summary from d facts, where d is drawn from a configurable range such as 1–20. It produces a mixed stream of corrections and queries. Corrections arrive at random times with a set average rate (a Poisson process), with a configurable mix of the four change types, and each summary is queried according to its cold, warm, or hot group from Section III. In *shifting* workloads, summaries are periodically moved between groups to simulate changes in what users ask about; this is varied separately from how often corrections arrive. GPT-4o-mini writes the facts, summaries, and query templates in natural language once, and our code then fills in the templates, which keeps generation affordable at 100K–1M scale. Every query comes with its correct answer, computed from the ledger at the time of the query, so we can tell exactly when an answer is stale.

G. Implementation Platform

The system is written in Python 3.11. The ledger and summary store use SQLite, and DuckDB is used to analyze experiment logs. Meaning-based search uses FAISS, BGE-M3 runs through the FlagEmbedding library, and the 8B model is served locally through vLLM (or llama.cpp on Colab if vLLM is unavailable). Policies are trained with XGBoost or LightGBM and scikit-learn. Experiments run in a simulator that replays corrections and queries in time order, with settings stored in YAML files and fixed random seeds so every run can be repeated exactly.

The simulator has two modes that use the same policy and staleness code. In *LLM-in-the-loop* mode, every VERIFY and REPAIR actually calls the model; we use this at the 10K scale and on the real-history benchmarks. In *cost-replay* mode, the model is not called; instead, the cost of each operation is drawn from the costs we recorded in LLM-in-the-loop runs. This lets us test policy decisions at 100K and 1M components without making millions of LLM calls.

V. Validation

A. Testing

Every component has unit tests written with `pytest`, which run automatically on each commit through GitHub Actions. The same test suite serves as the regression check in the Week 12 artifact audit.

Ledger tests check that no record is ever edited or deleted, that after any change there is exactly one currently valid record for each entity and attribute, and that looking up a fact at any past time returns the right version for all four change types, including the old value coming back after a temporary exception ends. For the summary store, we run randomized tests that build many random sets of facts and summaries, apply random sequences of corrections and repairs, and check that the fact-to-summary lookup table always matches each summary's `source_facts` list. Staleness tests check that every summary depending on a corrected fact, directly or through other summaries, is marked stale.

For the maintenance operations, tests check that VERIFY never changes the stored summary, and that REPAIR clears the stale flag, records the current fact versions, and updates both search indexes. Policy tests check that P1 repairs on exactly the $\lceil c_r/c_v \rceil$-th query of an episode, that no policy spends more than its repair budget, and that P2 and P3 make the same choices as the oracle when given the true k_s. The oracle itself is checked by trying every possible sequence of actions on small hand-built examples. Runs with the same seed and settings must produce identical results, and a toy dataset of about 100 facts and 50 summaries is run end to end against known answers.

The security part of the audit covers three things: scanning our software dependencies for known vulnerabilities with `pip-audit`, checking that no API keys (such as the one for the data generator) are committed to the repository, and a prompt-injection test in which fact values contain instruction-like text (e.g., "ignore previous instructions"), to confirm that VERIFY and REPAIR treat them as plain data rather than commands.

B. Performance Metrics

For correctness, we report the stale-answer rate, the fraction of answers that match an outdated value in the ledger, and overall answer quality. On synthetic workloads, answer quality is measured by exact match and token F1 (how much the wording overlaps with the correct answer); on the real-history benchmarks, we use the official metrics of LongMemEval and RealTime QA. For efficiency, we report the number of verifications, the amount of repair work (number of repairs and tokens generated), and total cost $C(\pi)$ from Eq. (1). For speed, we report the median (p50), 95th percentile (p95), and 99th percentile (p99) query latency; the last two show how slow the slowest queries are. We report these separately for

queries that did and did not trigger maintenance. For decision quality, we report regret and the ratio $C(\pi)/C(\text{oracle})$, which says how many times more a policy spends than the oracle, broken down by query group (cold, warm, hot). Finally, we report flag precision: the fraction of summaries marked stale that the semantic dirty-bit check confirms really needed fixing.

C. Experimental Design

We vary the cost ratio ρ, how often corrections arrive, the number of facts per summary d, the mix of cold, warm, and hot summaries, and the repair budget B. We run these on stable and shifting workloads at 10K and 100K components, and at 1M in cost-replay mode if time permits. Each setting is run with five different random seeds, and we report the average with 95% confidence intervals. Learned policies are always tested on different traces than they were trained on, and shifting-workload tests use group assignments the model never saw during training.

We test three hypotheses. H1: the simple threshold rule P1 never costs more than always repairing (Eager Refresh) or never repairing (Verify-Only), whatever the mix of summaries, and does not give more stale answers. H2: the learned policy P3 comes closer to the oracle than P1 on warm and hot summaries, where knowing future usage matters most. H3: when usage patterns change, the hedged version of P3 stays close to the oracle, while the version that fully trusts its predictions falls further and further behind.

VI. Deliverables

The deliverables are split into three tiers. Each tier builds on the previous one, and each is a usable result on its own if later tiers are not finished.

Tier 1: Minimum (by the midterm, Week 9).

- The memory system: the fact ledger, the summary store with its fact-to-summary lookup table, staleness marking, and hybrid BGE-M3 search, along with its tests.
- The synthetic workload generator, with control over the types of change and how often summaries are queried.
- The P1 and P2 threshold policies compared with Eager Refresh, Verify-Only, and Repair-on-First-Access at 10K, with cost and p95 latency results.

Tier 2: Expected (by the final report, Week 15).

- All baselines, the semantic dirty-bit check, and the hindsight oracle, with results on how far each policy is from the oracle.
- The learned policy P3, with an analysis of which inputs most influence its repair decisions.
- Repair-budget experiments and stable vs. shifting workload results at 10K.
- Answer-quality results on the LongMemEval and RealTime QA subsets.
- Scaling results at 100K.
- A GitHub repository that others can use to reproduce our results, and the final paper.

Tier 3: Stretch.

- Results at 1M components in cost-replay mode, with an analysis of where the system slows down.
- The hedged version of P3, tested on workloads that change in ways designed to mislead its predictions.
- A public release of our correction-and-query traces as a benchmark for memory maintenance.

VII. Timeline

See Table I.

References

[1] N. Yadav, "Temporal Validity in Retrieval Memory: Eliminating Stale-Fact Errors for AI Agents over Evolving Knowledge," arXiv:2606.26511, 2026.

[2] C.-C. Lv, H. Chang, Y. Guo, S. Tao, and S. Zhou, "All-Mem: Agentic Lifelong Memory via Dynamic Topology Evolution," arXiv:2603.19595, 2026.

[3] Vectorize, "Staleness-Aware Memory: When Your Agent Should Verify Before It Trusts," Hindsight Documentation, 2026.

[4] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef, "Zep: A temporal knowledge graph architecture for agent memory," arXiv:2501.13956, 2025.

[5] A. R. Karlin, M. S. Manasse, L. Rudolph, and D. D. Sleator, "Competitive snoopy caching," *Algorithmica*, vol. 3, pp. 79–119, 1988.

[6] A. R. Karlin, M. S. Manasse, L. A. McGeoch, and S. Owicki, "Competitive randomized algorithms for nonuniform problems," *Algorithmica*, vol. 11, no. 6, pp. 542–571, 1994.

[7] M. Purohit, Z. Svitkina, and R. Kumar, "Improving online algorithms via ML predictions," in *Proc. NeurIPS*, 2018.

[8] Z. Li et al., "Combinatorial Ski Rental Problem: Robust and Learning-Augmented Algorithms," in *Advances in Neural Information Processing Systems 38*, 2025.

[9] K. Meng et al., "Locating and Editing Factual Associations in GPT," in *Advances in Neural Information Processing Systems 35*, 2022, pp. 17359–17372.

[10] K. Meng et al., "Mass-Editing Memory in a Transformer," in *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.

[11] J. Shi et al., "EchoEdit: Consistent multi-hop question answering via ripple control in knowledge editing," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 23, 2026, pp. 19362–19370.

[12] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, "LongMemEval: Benchmarking chat assistants on long-term interactive memory," in *Proc. ICLR*, 2025.

[13] D. Wu et al., "LongMemEval-V2: Evaluating Long-Term Agent Memory Toward Experienced Colleagues," arXiv:2605.12493, 2026.

[14] J. Kasai *et al.*, "RealTime QA: What's the answer right now?" in *Proc. NeurIPS Datasets and Benchmarks Track*, 2023.

[15] A. Grattafiori *et al.*, "The Llama 3 herd of models," arXiv:2407.21783, 2024.

[16] Qwen Team, "Qwen2.5 technical report," arXiv:2412.15115, 2024.

[17] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," in *Findings of ACL*, 2024.

[18] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. KDD*, 2016.

[19] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Proc. NeurIPS*, 2017.

[20] J. Lin *et al.*, "AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration," in *Proc. MLSys*, 2024.

[21] W. Kwon *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. SOSP*, 2023.

[22] M. Douze *et al.*, "The Faiss library," arXiv:2401.08281, 2024.

[23] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. SIGIR*, 2009.

TABLE I
PROJECT TIMELINE AND DELIVERABLES

Weeks	Core Tasks & Milestones	Key Deliverables
1–2	Group formation and project selection.	Final customized project idea.
3–5	Finalize proposal and set up the GitHub repository. Implement the Source Fact Ledger, Derived Summary Store, and hybrid search. Build the Eager Refresh and Verify-Only baselines.	Completed proposal, active repository, and a small demo of search that shows which facts each summary depends on. First cost measurements for the baselines.
6–7	Implement the correction stream, staleness marking, and the remaining baselines (Periodic Refresh, Repair-on-First-Access, Oldest-First).	Working correction pipeline, stale marks that pass through chains of summaries, and baseline comparisons on the toy dataset.
8–9	Scale the synthetic workload generator to 10K memory components. Implement the accumulated threshold policy (P1) and the predicted threshold policy (P2). Re-run the baselines. Finalize midterm report.	10K-scale generator, p95 latency and cost plots, and results comparing P1 and P2 to the baselines. Midterm report.
10–11	Build the hindsight oracle and the semantic dirty-bit check. Design inputs for and train the learned policy (P3). Add the repair budget and run experiments with limited budgets.	Oracle and regret results, false-alarm (flag precision) results, comparison of P3 with P1 and P2, and cost results under different repair budgets.
12	Add workloads with different correction rates and with changing usage patterns (summaries moving between groups). Vary the number of facts per summary. Complete artifact audit (peer review, regression check, security test, and release rehearsal).	Policy results on stable, frequently corrected, and shifting workloads at the 10K scale. Artifact audit.
13–14	Add LongMemEval and RealTime QA to measure answer quality alongside cost. Move to the local GPU or HPC cluster to test the 100K scale. Find where the system slows down, clean up the repository, and draft the paper.	Answer-quality results, 100K scaling results, fully reproducible GitHub repository, and paper draft.
15	Polish the final paper and create slides for the in-class presentation.	Final research paper and presentation slides.

APPENDIX

A. Combinatorial Ski Rental Problem: Robust and Learning-Augmented Algorithms

Li et al. [8] tackle a harder version of the classic ski rental problem. Instead of deciding whether to rent or buy a single item, the decision-maker faces several items (skis, boots, helmet) that can be rented separately or bought individually or as a discounted bundle, all without knowing how long the season will last. The authors call this the Combinatorial Ski Rental (CSR) problem, and it models real choices like picking software bundles or cloud service packages. Their main technique is a randomized algorithm called SOAC (Sorted Optimal Amortized Cost), which they prove achieves the best possible competitive ratio, meaning no other online algorithm can beat its worst-case cost relative to an all-knowing decision-maker. SOAC also extends to related variants, such as multi-slope, multi-shop, and multi-commodity ski rental. Building on SOAC, they add machine-learned predictions (for example, a forecast of the season's length) to create a learning-augmented algorithm that performs well when predictions are accurate (consistency) but still protects against bad ones (robustness).

The most important results are the optimality proof for SOAC and the fact that the learning-augmented version matches or improves on earlier results for classic and multi-shop ski rental. Its main limitation is that SOAC's computation can grow exponentially with the number of bundle combinations, though this is usually manageable in practice since few combinations are discounted.

For our project, this paper provides the theoretical backbone for the "verify (rent) vs. repair (buy)" decision. Its consistency and robustness framing fits our demand-aware policy, which would use predicted query frequency the way this paper uses predictions, while keeping a worst-case guarantee if the prediction is wrong. Since our setting involves many linked memory components, the combinatorial angle (repairing related memories together may cost less than repairing them separately) could be a natural connection, though that is our interpretation and not something the paper claims.

B. Locating and Editing Factual Associations in GPT / Mass-Editing Memory in a Transformer

Meng et al. [9], [10] address the problem that large language models learn from a fixed snapshot of data, so their knowledge goes out of date as the world changes. If a model still thinks a former officeholder is in office, we need a way to correct it. Retraining is far too expensive to do for every change, and fine-tuning on new facts risks catastrophic forgetting, where learning one thing overwrites something the model already knew. This is the same underlying problem as in our project: what should a system do once it learns that something it stored is wrong, when fixing everything is costly? ROME and MEMIT are two answers to this problem for the model's internal knowledge, and comparing them shows how the right fix depends on how many corrections you need to make.

Both papers start from the idea that facts are stored in the feed-forward layers of a Transformer, which work like a lookup table: a subject (like "Eiffel Tower") is the key, and the fact ("Paris") is the value. ROME uses this to find the one matrix where a fact lives and applies a tiny, targeted change (a rank-one update) to rewrite it. It is precise, and the edit holds up when the question is reworded or asked in

another language. The catch is that it is built for one edit at a time. When many edits are applied in sequence, they start to interfere with each other and the model gradually degrades. MEMIT keeps the same core idea but changes the strategy. It gathers thousands of new facts, computes one combined update that resolves conflicts between them, and spreads the change across several layers instead of piling it into one. The most important result is that MEMIT lets us go from correcting a single fact to correcting thousands without the slow damage that repeated ROME edits cause. Both methods share a limitation, though. They fix the specific fact but not everything that depends on it, so correcting who holds an office may leave facts about that person's family unchanged.

We can use ideas from ROME and MEMIT in two ways. From MEMIT, we can batch repairs so that memories affected by the same correction are fixed together rather than one at a time, which should reduce total repair cost. From both methods' shared limitation, we learn that fixing one fact does not fix everything that depends on it, so we can use dependency links to flag every affected memory. Since permanent edits carry costs and risks, we can also treat repair as something to use only when a memory is queried often enough to justify it, and use verify otherwise.

C. All-Mem: Agentic Lifelong Memory via Dynamic Topology Evolution

Lv et al. [2] tackle the problem of retrieving the right evidence in an agentic system designed for the long term. Existing memory systems accumulate redundant and outdated information over the long-term, and prior systems struggle with balancing memory optimization and real-time latency. This paper introduces All-Mem, an online/offline system that treats memory as a graph and does most of the memory reorganization in the background. The online phase executes lightweight writes (1–2 sentence summary and keywords generated by an LLM in a single call), performs a small search operation over visible memory nodes, creates temporary graph links, and logs IDs to a buffer, while not performing any memory restructuring operations. The background offline phase uses an LLM to diagnose the memory structure and propose SPLIT, MERGE, and UPDATE graph operations and applies them to the graph, pruning proposals below a specified confidence threshold. The memory is never deleted, but it is archived by setting a visibility flag to false (still discoverable by the agent through hop-bounded expansion along typed links, particularly version links). All-Mem outperformed baseline systems such as Mem0 and A-Mem in evidence retrieval and query answering accuracy, as well as significantly reducing query-time latency while not compromising accuracy or reachability.

This is useful and relevant to our project because it shows us a way to model our memory structure after. We can consider modelling our memory as a typed graph and performing splitting, merging, and updating when our model decides to "repair". Additionally, this gives us a state-of-the-art comparison baseline that will allow us to evaluate how our on-demand verify-or-repair decisions perform against scheduled offline reorganization in a constrained budget. Finally, we can also look into leveraging offline processes and maybe caching.

D. EchoEdit: Consistent Multi-hop Question Answering via Ripple Control in Knowledge Editing

Shi et al. [11] tackle the "ripple effect" problem when editing LLM knowledge, where the update of one fact is not reflected by the model when given a related question. The paper introduces EchoEdit, a framework designed to control how new knowledge spreads without modifying the model's underlying weights. The main idea begins with creating a "RippleGraph", which is a graph of semantically relevant nodes centered around the edited fact, pulling from both external knowledge and internal memory. Then, the RippleRule Generator is a dynamic rule generator that reorganizes the graph to update outdated, conflicting, or irrelevant information, creating a more compact/dense RippleGraph. Then, EchoEdit uses a Chain-of-Thought planner to break down complex questions into sub-problems that go through the updated RippleGraph. The most important results are its significantly high accuracy and reasoning stability.

This paper is useful for our project because it provides a solution to clearing the gap between a simple fact "repair" and a robust ripple update. When our model decides to repair, we can use the idea of a RippleGraph with RippleRules to generate and compact a graph of semantically similar ideas, and repair them all.

E. LongMemEval-V2: Evaluating Long-Term Agent Memory Toward Experienced Colleagues

Wu et al. [13] explore whether an AI agent can recall facts about how an environment behaves from past trajectories of its interactions with the environment. The paper proposes a benchmark to test the efficacy of an agent in recalling specifics of an environment and tests two methods for storing and searching the agent's history. For their benchmark, the authors wrote 451 questions about web environments to test the agent's ability to provide a correct answer given Small or Medium trajectory histories, where the true answer is only contained in a small part of that history. They evaluated the agent's answer against a fixed reader model to see which agent answered more accurately and took less time for retrieval. They then propose two methods: AgentRunbook-R which is a retrieval system that stores memories in three ways: raw observations nearby, then records of what changed after the actions, and farther away, high-level notes about procedures and possible pitfalls when interacting with the environment. AgentRunbook-C stores the trajectories as files with a workflow document, a map of available files, and helper functions for viewing elements of a trajectory. The best performance came from AgentRunbook-C but it also took quite a bit longer. This paper is relevant to the domain in terms of showing how other researchers explored this same problem of memory storage and retrieval, which we can build upon.

F. Improving Online Algorithms via ML Predictions

Purohit et al. [7] present an advancement on the ski rental problem discussed above by other papers. While the classic strategy is to rent the skis until the accumulated rental cost is approximately equal to the purchase cost, this paper introduces an ML predictor which estimates the number of skiing days. The authors then use this prediction to move the buying threshold by introducing an additional parameter that they multiply to the current strategy variables. This helps nudge the rent-or-buy problem using their prediction. They also explore a secondary problem domain of computer job scheduling. They use the same strategy of predicting how long a job will take to then perform informed scheduling. This paper informs us of different strategies used in rent-or-buy problems like the one we are trying to solve and can provide inspiration for our approach.